

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

JavaScript

CIS 2102 Review

DCISM
DEPARTMENT OF COMPUTER, INFORMATION SCIENCES AND MATHEMATICS

Topics

1. Introduction
2. Basics
3. JavaScript Objects
4. DOM
5. Event Handling

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Introduction

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- JavaScript was introduced in 1995 to add programs to web pages in the Netscape Navigator browser.

- It has made modern web applications possible; applications with which you can interact directly without doing a page reload for every action. JavaScript is also used in more traditional websites to provide various forms of interactivity and cleverness.

- JavaScript has almost nothing to do with the programming language named Java.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Web browsers are not the only platforms on which JavaScript is used. Some databases, such as MongoDB and CouchDB, use JavaScript as their scripting and query language. Several platforms for desktop and server programming, most notably the Node.js project, provide an environment for programming JavaScript outside of the browser.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

ECMAScript Standard

- Standard document written to describe the way the JavaScript language should work on various pieces of software that claimed to support JavaScript. Standardized by ECMA International (European Computer Manufacturers Association)

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

JavaScript is a weakly typed / untyped language

- Although the term "*weakly typed*" might imply some deficiency with JavaScript, the term actually means that JavaScript is smart enough to do the work of determining what type of data any given variable should be.

- In short, JavaScript (and not you) does most of the work in figuring out data types.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

How to use JavaScript?

- It can be embedded in HTML by:
  - Script block - Usually found inside the HTML file

```
<script type="text/javascript">
        …
</script>
```

  - Linking JS file - can also be saved in an external .js file which can be linked through

```
<script type="text/javascript" src= "path" >
</script>
```

  - You can also place an external script reference in `<head>` or `<body>`.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Basics

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Variables

- JavaScript variables can be defined using ***`var`***, ***`let`*** and ***`const`*** as keywords.
- ***`var`***: The scope of the variables defined by this is limited to the function it is declared. If it is declared outside any function, the scope of such variable will be global. (*var is a function scoped variable*)
- ***`let`*** *&* ***`const`***: The scope of the variables defined by either is limited to the block which is defined by {}. However when a variable is defined by ***const***, it cannot be reassigned (*let & const are block scoped variables*).

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
function foo(){
    if(true){
        var var1 = 1;
        const var2 = 2;
        let var3= 3;
    }
    console.log(var1);
    console.log(var2);
    console.log(var3);
}
```

Output:

?

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
function foo(){
    if(true){
        var var1 = 1;
        const var2 = 2;
        let var3= 3;
    }
    console.log(var1);
    console.log(var2);
    console.log(var3);
}
```

Output:

```
1
var2 is not defined
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Naming variables

- Can contain numbers but must not start with them
- Can include $ or _ but no other punctuation or special characters
- Don’t use keywords or reserved words as variable names

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Keywords

break case catch class const continue debugger default delete do else enum export extends false finally for function if implements import interface in instanceof let new package private protected public return static super switch this throw true try typeof var void while with yield

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Data types

- **Number** – represents both integer and floating point numbers. There are also so-called “special numeric values” which also belong to this data type: Infinity, -Infinity and NaN.
  - **Infinity**: represents the mathematical Infinity ∞. It is a special value that’s greater than any number. We can get it as a result of division by zero or directly assigning the value Infinity:
  - **NaN**: represents a computational error. It is a result of an incorrect or an undefined mathematical operation.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- **BigInt** – since integer numbers has a limit (can't represent larger than 2^{53} or less than -2^{53}), JavaScript provided the BigInt data type. It is done by appending "n" at the end of an integer.
- **String** – must be surrounded by either *"double quotes"*, *'single quotes'* or *`backticks`*. Double and single quotes are simple quotes while backticks are extended functionalities. This allows for variables to be embedded in the string.

  Syntax:

```
let name = "John Doe";
alert(`Hello my name is ${name}`);
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- **Boolean** – has 2 values: true or false
- **Null** – *null* represents nothing, empty or unknown value
- **Undefined** – *undefined* value represents “value not assigned”

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Operators

- **Operand** – what operators are applied to

  Sample: 1 + 2

  1 and 2 are called operands

- **Unary Operators** – represented by having only a single operand
- **Binary Operators** – represented by having 2 operands

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Arithmetic Operators

- Addition (+) – This also serves as string concatenation if you use strings.
- Subtraction (-)
- Division (/)
- Remainder (%)
- Multiplication (*)
- Exponentiation (**)
- Increment (++) – This can be used as prefix or postfix
- Decrement (--) – This can be used as prefix or postfix

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
let a = 1, b = 1;

console.log ( ++a );
console.log ( b++ );

console.log ( a );
console.log ( b );
```

Output:

?

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
let a = 1, b = 1;

console.log ( ++a );
console.log ( b++ );

console.log ( a );
console.log ( b );
```

Output:

```
2
1

2
2
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Unary negation (-) – negates the value of an operand
- Unary plus (+) – converts a non-number operand to a number type
- Assignment (=)

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Comparison Operators

- == equal to
- === equal value **and** equal type
- != not equal
- !== not equal value **or** not equal type
- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
var var1 = (10 / 2) == "5";
var var2 = 11 === "11";
var var3 = 100 != "100";
var var4 = 57 !== "58";

console.log(var1);
console.log(var2);
console.log(var3);
console.log(var4);
```

Output:

?

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
var var1 = (10 / 2) == "5";
var var2 = 11 === "11";
var var3 = 100 != "100";
var var4 = 57 !== "58";

console.log(var1);
console.log(var2);
console.log(var3);
console.log(var4);
```

Output:

```
true
false
false
true
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Logical Operators

- && and
- || or
- | not

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Ternary Operator

also called as a conditional operator

```
var obj = (condition) ? value1 : value2;
```

similar to

```
if (condition){
      var obj = value1;
}else{
      var obj = value2;
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Bitwise Operators
(rarely used)

- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- LEFT SHIFT (<<)
- RIGHT SHIFT (>>)
- ZERO-FILL RIGHT SHIFT (>>>)

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Functions

a set of statements that performs a task

- A function is created with an expression that starts with the keyword *function*.
- Functions have a set of parameters and a body, which contains the statements that are to be executed when the function is called. It is enclosed in curly braces.
- Functions can also be assigned to a variable.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Syntax:

```
const square = function(x) {
      return x * x;
};

or

function square(x){
      return x * x;
}

console.log(square(12));
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Arrow function

Syntax:

```
const variable = (parameter) => {
      …
};

variable (parameter)
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Alert function

- Alert function resembles a prompt, in that it pops up a little window, but only shows a message without asking for input.

Syntax:

```
alert("string");
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

I/O functions

- Output to console:

```
console.log(string);
```

- Input:

```
var obj = prompt("string");
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Conditional Statements

- *if* statement specifies a code block for execution if condition is true.
- *else* statement specifies a code block for execution if condition is false.
- *else if* statement specifies a code block for execution if the first condition is false.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Syntax:

```
if (condition) {
        …
} else if (another condition) {
        …
} else {
        …
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Switch – case
  - Used to perform different actions based on different conditions.
  - Parts:
    - *expression* can be a number or string.
    - *case* contains the value of the expression. There should be no duplicate case values.
    - *default* is optional. It is only executed if no case value matches.
    - *break* statement is optional. It is used to end or break from the switch block. If omitted, the next case will be executed

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Syntax:

```
switch(expression) {
      case value:
            body;
            break;
      case value2:
            body;
            break;
      default:
            body;
            break;
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Sample:

```
var age = prompt("How old are you?")
switch(age){
        case 10:
                console.log("Young");
                break;
        case 30:
                console.log("Adult");
                break;
        case 60:
                console.log("Senior");
                break;
        default:
                console.log("Input again");
                break;
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Pretest Loops

condition is checked before each repetition to determine if the loop should terminate

- **for loop**:
  - Syntax

```
for (statement 1; condition; statement 2) {
        code block
}
```

  - statement1 is executed before code block is executed
  - statement2 is executed after the block is executed

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- **for…in loop:**
  - Iterates over all properties of an object

    Sample:

```
const object = {a: 1, b: 2, c: 3};
for (var variable in object) {
        console.log(`${variable }: ${object[variable]}`);
}
```

- **for…of loop**:
  - Use to loop through, arrays, strings, etc.

    Sample:

```
for (var variable of iterable) {
        code block
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- **while loop:**
  Syntax:

```
while(condition){
        …
}
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Postest Loops

condition is checked after each repetition for termination

- **do…while loop:**

  Syntax:

```
do{
        code block
} while(condition);
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Arrays

List-like objects that can hold more than one value at a time of the same type.

- Creating array:

```
let arr = [value1, value2];
let arr = new Array(value1, value2);
let arr = new Array(length);
```

- Accessing an item:

```
console.log(arr[index]);
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Iterating through:

```
for (var variable of arr) {
      console.log(variable);
}
```

- Adding an item at the end:

```
arr.push(value3);
```

- Remove from the end:

```
arr.pop();
```

- Remove from the front:

```
arr.shift();
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Add to the front:

```
arr.unshift(value4);
```

- Find the index of an item in the Array:

```
let pos = arr.indexOf(value2);
```

- Remove an item by index position:

```
arr.splice(index, number);
```

- Copy an array:

```
let shallowCopy = arr.slice();
```

- Get array length:

```
var len = arr.length;
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Iterating through an array

```
let arr = [1, 2, 3, 4, 5];
arr.map((x)=>{
    console.log(“Number: ” + x);
})
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

JavaScript Objects

are arbitrary collections of properties which are written as key : value pairs



UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Creating objects

- One way to create an object is by using braces as an expression.

  Syntax:

```
let car = {
      brand: "Toyota",
      model: "Vios",
      variants: ["J", "XE", "XLE", "E", "G"]
};
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Using the new keyword

  Syntax:

```
var car = new Object();
car.brand = "Toyota";
car.model = "Vios";
car.variants = ["J", "XE", "XLE", "E", "G"];
```

- When an object is written over multiple lines, indenting it like in the example helps with readability.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Properties whose names aren't valid identifiers have to be quoted.

```
let descriptions = {
      work: "Went to work",
      "touched tree": "Touched a tree"
};
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Calling object properties

- `objectName.property`
- `objectName["property"]`

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Adding object properties

- Adding new properties to an existing object is done by simply giving it a value.

  Syntax:

```
objectName.newProperty = value;
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Deleting object properties

- Done by using the *delete* keyword
- Deleting means removal of both property and value

Syntax:

```
delete objectName.property;
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

DOM

Document Object Model

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- When you open a web page in your browser, the browser retrieves the page's HTML text and parses it.

- (Parsing reads a piece of text and produces a data structure that reflects the structure of the program contained in that text. If the text does not form a valid program, the parser should point out the error.)

- The browser builds up a model of the document's structure and uses this model to draw the page on the screen.

- The **DOM** is an object-oriented representation of the web page, which can be modified with a scripting language such as JavaScript.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

DOM Manipulation

- `var obj = document.getElementsByTagName("tag_name")[index];`
- `var obj = document.getElementById("id");`
- `var obj = document.getElementsByClassName("class_name") [index];`
- `var obj = document.querySelector("selector");`
- `var obj = document.querySelectorAll("selector");`

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- ```
  var obj = document.createElement(tagName);
  document.body.appendChild(obj);
  ```

- Applying style

  obj.style.property = "new property"

  ```
  obj.style.color = "red";
  ```

- Changing Attributes:

  obj.attribute = "value";

  ```
  obj.placeholder = "Enter name";
  ```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO
  ```

getAttribute() and setAttribute()

- Some element attributes, such as href, src, height, width, style and other common attributes can be accessed through a property of the same name on the element's DOM object.

- But since HTML allows you to set any attribute you want; your own attribute names will not be present when accessed directly.

- Instead of directly accessing the attribute, you have to use the *getAttribute()* and *setAttribute()* methods.

- Made-up attributes are recommended to prefix with data- to prevent conflict with other attributes.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Syntax

- getAttribute("attribute")

```
let image = document.getElementById("imageUser");
let source = image.getAttribute("src");
```

- setAttribute("attribute", "value")

```
let link = document.getElementById("defaultLink");
link.setAttribute("href", "www.google.com");
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- Changing class

```
obj.className = "class";
obj.classList.add("new class");
obj.classList.remove("new class");
obj.classList.toggle("class");
obj.classList.contains("class");
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Removing an element

- **remove()**

```
var element = document.getElementById("id");
element.remove();
```

- **removeChild()**

```
var parent = document.getElementById("parentID");
var child = document.getElementById("childID");
parent.removeChild(child);
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Event Handling

An **event** is a signal that something has happened. All DOM elements generate such signals .

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Sample events

- **onload** and **onunload** events are triggered when user enters or leaves a page.
- **onchange** event is often used in input fields.
- **onmouseover** and **onmouseout** events can be used to trigger a function when the user's mouse is over or out of the HTML element.
- When a mouse-button is clicked, the **onmousedown** event is triggered.
- When the mouse-button is released, the **onmouseup** event is triggered.
- When the mouse-click is completed, the **onclick** event is triggered.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Event handlers

- To react on ***events*** we can assign a function(handler) that runs in case of an event.

- *Handlers* are a way to run JavaScript code in case of user actions.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Syntax

```
<button type="button" onclick="helloWorld()">Click Me</button>
…
<script type="text/javascript" >
      function helloWorld() {
            alert("Hello, world!");
      }
</script>
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

```
<button type="button">Click Me</button>


<script type="text/javascript" >
    function function_name(){
       …
    }

    function readyFunction(){
       var obj = document.querySelector('button’);
       //add event listener to HTML element
       obj.addEventListener('click', function_name);
       //remove event listener to HTML element
       obj.removeEventListener('click', function_name);
    }
</script>
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

Event objects

- Though it has been ignored, event handler functions are passed with an argument called *event objects*.

- This holds information about the event.

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

- For example, we want to know what mouse button was pressed, we can check the ***button*** property in the event object.

- The information stored in an event object differs per type of event.

```
<button onclick= "buttonClicked(event)" >Click</button>
…
<script type= "text/javascript">
      function buttonClicked(e){
            console.log(e);
      }
</script>
…
```

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

References

- https://javascript.info/
- https://eloquentjavascript.net/